

Licence Informatique, Réseaux & Multimédia

Collecte de Données Multimodales



Réalité Logistique

Guide Complet des Mécanismes d'Accès aux Données



JSON APIs



Zyte API

Réalisé par :

Prof. ABDELLAH ADIB



Requests



Apollo Client



aiohttp



Playwright



BeautifulSoup



Scrapy

XPath



Scapgraph AI



Crawl4ia

Contents

1	Introduction	4
1.1	Contexte et Objectifs	4
1.2	La Réalité Logistique	4
1.3	Vue d'Ensemble des Mécanismes	4
2	Téléchargement HTTP/HTTPS	5
2.1	Principe Fondamental	5
2.1.1	Architecture Client-Serveur	5
2.1.2	Méthodes HTTP Principales	5
2.1.3	Codes de Statut HTTP	5
2.2	Implémentation avec Python	6
2.2.1	Installation de la Bibliothèque Requests	6
2.2.2	Téléchargement Simple d'un Fichier	6
2.2.3	Téléchargement avec Barre de Progression	7
2.2.4	Téléchargement avec Headers Personnalisés	8
2.3	Téléchargement Asynchrone	8
2.3.1	Installation des Dépendances	8
2.3.2	Implémentation Asynchrone Complète	8
2.4	Gestion des Sessions et Cookies	10
2.5	Bonnes Pratiques	11
3	APIs REST	12
3.1	Principe et Architecture	12
3.1.1	Caractéristiques d'une API REST	12
3.1.2	Structure Typique des URLs	12
3.2	Authentification	12
3.2.1	Types d'Authentification	12
3.3	Implémentation Complète d'un Client API	13
3.4	Exemples d'APIs Populaires	17
3.4.1	API OpenWeatherMap	17
3.4.2	API GitHub	17
4	APIs GraphQL	20
4.1	Introduction à GraphQL	20
4.1.1	Avantages par rapport à REST	20
4.1.2	Types de Requêtes	20
4.2	Syntaxe des Requêtes GraphQL	20
4.2.1	Requête Simple	20
4.2.2	Requête avec Relations	21
4.2.3	Requête avec Variables	21
4.3	Implémentation Python	22
4.3.1	Client GraphQL Simple	22

4.3.2	Requêtes avec Variables	23
4.3.3	Mutations GraphQL	24
4.4	Introspection du Schéma	25
5	Web Scraping	27
5.1	Introduction au Web Scraping	27
5.1.1	Cas d'Utilisation Légitimes	27
5.1.2	Considérations Éthiques et Légales	27
5.2	Scraping Statique avec BeautifulSoup	28
5.2.1	Installation	28
5.2.2	Concepts Fondamentaux	28
5.2.3	Sélecteurs CSS	28
5.2.4	Scraper Complet avec Gestion d'Erreurs	29
5.3	Scraping Dynamique avec Selenium	32
5.3.1	Installation	32
5.3.2	Configuration du Driver	32
5.3.3	Attente des Éléments	33
5.3.4	Scraper Dynamique Complet	34
5.4	Scraping à Grande Échelle avec Scrapy	37
5.4.1	Installation	37
5.4.2	Structure d'un Projet Scrapy	37
5.4.3	Définition des Items	37
5.4.4	Spider Complet	38
5.4.5	Configuration (settings.py)	39
5.4.6	Pipelines de Traitement	39
5.4.7	Exécution du Spider	41
5.5	Scraping Intelligent avec l'IA : ScrapeGraphAI	41
5.5.1	Principe et Avantages	42
5.5.2	Installation	42
5.5.3	Exemple Simple avec OpenAI	42
5.5.4	Utilisation avec Modèles Locaux (Ollama)	43
5.5.5	Types de Pipelines Disponibles	43
5.5.6	Scraping Multi-Pages	44
5.5.7	Génération Automatique de Scripts	44
5.5.8	Comparaison : Scraping Traditionnel vs IA	45
5.5.9	Bonnes Pratiques	45
5.5.10	Ressources	45
5.6	Crawl4AI : Crawler Open Source pour LLMs	46
5.6.1	Caractéristiques	46
5.6.2	Installation et Exemple	46
5.6.3	Extraction Structurée avec LLM	46
5.7	FireCrawl : API Cloud pour l'IA	47
5.7.1	Caractéristiques	47
5.7.2	Installation et Exemple	47
5.8	Jina Reader : Conversion Web Instantanée	48
5.8.1	Utilisation Sans Code	48
5.8.2	Utilisation avec Python	48
5.9	Spider.cloud : Le Plus Rapide	49
5.9.1	Exemple d'Utilisation	49
5.10	Comparaison des Outils IA de Scraping	49
5.10.1	Guide de Choix	50

6	Flux Temps Réel (WebSockets)	51
6.1	Introduction aux WebSockets	51
6.1.1	Cas d'Utilisation	51
6.1.2	Protocole WebSocket	51
6.2	Implémentation avec Python	51
6.2.1	Installation	51
6.2.2	Client WebSocket Simple	52
6.2.3	Client avec Reconnexion Automatique	52
6.2.4	Exemple : Flux Binance (Crypto)	55
6.2.5	Sauvegarde des Données en Temps Réel	56
7	Conclusion et Bonnes Pratiques	59
7.1	Récapitulatif des Mécanismes	59
7.2	Bonnes Pratiques Générales	59
7.3	Arbre de Décision	59
8	Bibliographie et Ressources	61
8.1	Ouvrages de Référence	61
8.2	Documentation Officielle	61
8.3	Tutoriels Recommandés	62
8.4	Sites d'Entraînement (Légaux)	62
8.5	APIs Publiques pour Pratiquer	62
8.6	Aspects Légaux et Éthiques	63

Chapter 1

Introduction

1.1 Contexte et Objectifs

La collecte de données constitue la première étape fondamentale de tout projet d'intelligence artificielle et d'apprentissage automatique. Sans données de qualité, même les algorithmes les plus sophistiqués ne peuvent produire de résultats pertinents. Ce rapport se concentre sur la **Réalité Logistique** de la collecte de données, c'est-à-dire l'ensemble des mécanismes permettant d'accéder à des données déjà numérisées et disponibles sur des infrastructures réseau.

1.2 La Réalité Logistique

La Réalité Logistique répond à la question fondamentale : *"Comment accéder aux données existantes ?"*

Cette dimension de la collecte se caractérise par :

- **Indépendance du contenu** : Les mêmes protocoles et techniques s'appliquent quel que soit le type de données (texte, images, audio, vidéo).
- **Abstraction du transport** : L'accent est mis sur les mécanismes d'acheminement plutôt que sur la nature des données transportées.
- **Standardisation** : Utilisation de protocoles normalisés (HTTP, WebSocket, etc.) et d'interfaces documentées (APIs).

1.3 Vue d'Ensemble des Mécanismes

Ce rapport couvre en détail les cinq principaux mécanismes de collecte logistique :

1. **Téléchargement HTTP/HTTPS** : Accès direct aux ressources via URLs
2. **APIs REST** : Interfaces structurées avec authentification
3. **APIs GraphQL** : Requêtes flexibles et précises
4. **Web Scraping** : Extraction depuis le code HTML (statique et dynamique)
5. **Flux Temps Réel** : Connexions persistantes pour données continues

Chapter 2

Téléchargement HTTP/HTTPS



Figure 2.1: Bibliothèques Python pour les requêtes HTTP

2.1 Principe Fondamental

Le protocole HTTP (HyperText Transfer Protocol) constitue la base de toute communication sur le Web. Il définit un modèle requête-réponse où un client envoie une demande à un serveur qui retourne les données sollicitées.

2.1.1 Architecture Client-Serveur

Le téléchargement HTTP repose sur une architecture simple :

1. Le **client** (votre script Python) formule une requête HTTP
2. La requête est envoyée au **serveur** identifié par l'URL
3. Le serveur traite la requête et renvoie une **réponse**
4. La réponse contient un **code de statut** et éventuellement des **données**

2.1.2 Méthodes HTTP Principales

- **GET** : Récupération de données (lecture seule)
- **POST** : Envoi de données au serveur (création)
- **PUT** : Mise à jour de données existantes
- **DELETE** : Suppression de données
- **HEAD** : Récupération des en-têtes uniquement (vérification)

2.1.3 Codes de Statut HTTP

Les codes de statut indiquent le résultat de la requête :

- **2xx (Succès)** : 200 OK, 201 Created, 204 No Content
- **3xx (Redirection)** : 301 Moved Permanently, 302 Found, 304 Not Modified
- **4xx (Erreur Client)** : 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
- **5xx (Erreur Serveur)** : 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

2.2 Implémentation avec Python

2.2.1 Installation de la Bibliothèque Requests

La bibliothèque `requests` est le standard de facto pour les requêtes HTTP en Python :

```
1 pip install requests
```

2.2.2 Téléchargement Simple d'un Fichier

Voici un exemple complet de téléchargement d'un fichier avec gestion des erreurs :

```
1 import requests
2 import os
3 from pathlib import Path
4
5 def telecharger_fichier(url, dossier_destination="downloads"):
6     """
7     Telecharge un fichier depuis une URL et le sauvegarde localement.
8
9     Args:
10         url (str): URL du fichier a telecharger
11         dossier_destination (str): Dossier ou sauvegarder le fichier
12
13     Returns:
14         str: Chemin du fichier telecharge, ou None en cas d'erreur
15     """
16     # Creer le dossier de destination s'il n'existe pas
17     Path(dossier_destination).mkdir(parents=True, exist_ok=True)
18
19     # Extraire le nom du fichier depuis l'URL
20     nom_fichier = url.split("/")[-1].split("?")[0]
21     chemin_complet = os.path.join(dossier_destination, nom_fichier)
22
23     try:
24         # Envoyer la requete GET avec un timeout
25         response = requests.get(url, timeout=30)
26
27         # Verifier le code de statut
28         response.raise_for_status()
29
30         # Sauvegarder le contenu dans un fichier
31         with open(chemin_complet, "wb") as fichier:
32             fichier.write(response.content)
33
34     print(f"Telechargement reussi: {chemin_complet}")
35     print(f"Taille: {len(response.content)} octets")
```

```

36     return chemin_complet
37
38     except requests.exceptions.Timeout:
39         print(f"Erreur: Timeout lors du telechargement de {url}")
40         return None
41
42     except requests.exceptions.HTTPError as e:
43         print(f"Erreur HTTP {response.status_code}: {e}")
44         return None
45
46     except requests.exceptions.RequestException as e:
47         print(f"Erreur de connexion: {e}")
48         return None
49
50
51 # Exemple d'utilisation
52 if __name__ == "__main__":
53     url = "https://data.gouv.fr/exemple/dataset.csv"
54     telecharger_fichier(url)

```

2.2.3 Téléchargement avec Barre de Progression

Pour les fichiers volumineux, il est utile d'afficher la progression :

```

1  import requests
2  from tqdm import tqdm
3
4  def telecharger_avec_progression(url, chemin_sortie):
5      """
6      Telecharge un fichier avec affichage de la progression.
7
8      Args:
9          url (str): URL du fichier
10         chemin_sortie (str): Chemin de sauvegarde
11     """
12     # Requete en mode stream pour ne pas charger tout en memoire
13     response = requests.get(url, stream=True)
14     response.raise_for_status()
15
16     # Recuperer la taille totale depuis les en-tes
17     taille_totale = int(response.headers.get("content-length", 0))
18
19     # Telecharger par blocs avec barre de progression
20     with open(chemin_sortie, "wb") as fichier:
21         with tqdm(total=taille_totale, unit="B", unit_scale=True) as
           barre:
22             for bloc in response.iter_content(chunk_size=8192):
23                 fichier.write(bloc)
24                 barre.update(len(bloc))
25
26     print(f"Telechargement termine: {chemin_sortie}")
27
28
29 # Utilisation
30 telecharger_avec_progression(
31     "https://exemple.com/gros_fichier.zip",
32     "gros_fichier.zip"

```

33)

2.2.4 Téléchargement avec Headers Personnalisés

Certains serveurs requièrent des en-têtes spécifiques :

```

1 import requests
2
3 def telecharger_avec_headers(url, chemin_sortie):
4     """
5     Telecharge avec des en-tetes HTTP personnalisés.
6     """
7     headers = {
8         # Simuler un navigateur web
9         "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
10                        "AppleWebKit/537.36 (KHTML, like Gecko) "
11                        "Chrome/91.0.4472.124 Safari/537.36",
12         # Accepter différents formats
13         "Accept": "text/html,application/json,*/*",
14         # Langue preferee
15         "Accept-Language": "fr-FR,fr;q=0.9,en;q=0.8",
16         # Compression acceptee
17         "Accept-Encoding": "gzip, deflate, br",
18     }
19
20     response = requests.get(url, headers=headers, timeout=30)
21     response.raise_for_status()
22
23     with open(chemin_sortie, "wb") as f:
24         f.write(response.content)
25
26     return chemin_sortie

```

2.3 Téléchargement Asynchrone

Pour télécharger plusieurs fichiers simultanément, l'approche asynchrone est beaucoup plus efficace.

2.3.1 Installation des Dépendances

```

1 pip install aiohttp aiofiles

```

2.3.2 Implémentation Asynchrone Complète

```

1 import asyncio
2 import aiohttp
3 import aiofiles
4 from pathlib import Path
5 from typing import List, Tuple
6
7 async def telecharger_fichier_async(
8     session: aiohttp.ClientSession,
9     url: str,

```

```

10     dossier: str
11 ) -> Tuple[str, bool]:
12     """
13     Telecharge un fichier de maniere asynchrone.
14
15     Args:
16         session: Session aiohttp partagee
17         url: URL du fichier
18         dossier: Dossier de destination
19
20     Returns:
21         Tuple (url, succes)
22     """
23     nom_fichier = url.split("/")[-1].split("?")[0]
24     chemin = Path(dossier) / nom_fichier
25
26     try:
27         async with session.get(url) as response:
28             if response.status == 200:
29                 contenu = await response.read()
30                 async with aiofiles.open(chemin, "wb") as f:
31                     await f.write(contenu)
32                 print(f"OK: {nom_fichier} ({len(contenu)} octets)")
33                 return (url, True)
34             else:
35                 print(f"Erreur {response.status}: {url}")
36                 return (url, False)
37
38     except Exception as e:
39         print(f"Exception pour {url}: {e}")
40         return (url, False)
41
42
43 async def telecharger_multiple(
44     urls: List[str],
45     dossier: str = "downloads",
46     max_concurrent: int = 10
47 ) -> dict:
48     """
49     Telecharge plusieurs fichiers en parallele.
50
51     Args:
52         urls: Liste des URLs a telecharger
53         dossier: Dossier de destination
54         max_concurrent: Nombre max de telechargements simultanes
55
56     Returns:
57         Dictionnaire {url: succes}
58     """
59     # Creer le dossier
60     Path(dossier).mkdir(parents=True, exist_ok=True)
61
62     # Limiter le nombre de connexions simultanees
63     connector = aiohttp.TCPConnector(limit=max_concurrent)
64
65     # Timeout global
66     timeout = aiohttp.ClientTimeout(total=60)
67

```

```

68     async with aiohttp.ClientSession(
69         connector=connector,
70         timeout=timeout
71     ) as session:
72         # Lancer tous les telechargements
73         taches = [
74             telecharger_fichier_async(session, url, dossier)
75             for url in urls
76         ]
77         resultats = await asyncio.gather(*taches)
78
79     return dict(resultats)
80
81
82 # Exemple d'utilisation
83 if __name__ == "__main__":
84     urls = [
85         "https://exemple.com/fichier1.csv",
86         "https://exemple.com/fichier2.csv",
87         "https://exemple.com/fichier3.csv",
88         "https://exemple.com/fichier4.csv",
89         "https://exemple.com/fichier5.csv",
90     ]
91
92     # Lancer les telechargements
93     resultats = asyncio.run(telecharger_multiple(urls))
94
95     # Afficher le resume
96     succes = sum(1 for v in resultats.values() if v)
97     print(f"\nResume: {succes}/{len(urls)} fichiers telecharges")

```

2.4 Gestion des Sessions et Cookies

Pour maintenir une session authentifiée :

```

1  import requests
2
3  def telecharger_avec_session():
4      """
5      Utilise une session pour maintenir les cookies et connexions.
6      """
7      with requests.Session() as session:
8          # Configurer des headers par default
9          session.headers.update({
10             "User-Agent": "MonScript/1.0"
11         })
12
13         # Se connecter (les cookies seront conserves)
14         login_data = {
15             "username": "utilisateur",
16             "password": "motdepasse"
17         }
18         session.post("https://site.com/login", data=login_data)
19
20         # Telecharger des fichiers proteges
21         # (les cookies de session sont automatiquement inclus)

```

```
22     response = session.get("https://site.com/fichier_protege.pdf")
23
24     with open("fichier_protege.pdf", "wb") as f:
25         f.write(response.content)
```

2.5 Bonnes Pratiques

1. **Toujours utiliser des timeouts** pour éviter les blocages
2. **Gérer les erreurs** avec des blocs try/except appropriés
3. **Respecter les serveurs** avec des délais entre requêtes
4. **Utiliser des sessions** pour les requêtes multiples vers le même domaine
5. **Vérifier les codes de statut** avant de traiter les données
6. **Privilégier le streaming** pour les gros fichiers

Chapter 3

APIs REST



Figure 3.1: Technologies clés des APIs REST

3.1 Principe et Architecture

REST (Representational State Transfer) est un style d'architecture pour la conception d'APIs web. Une API REST expose des **ressources** accessibles via des URLs et manipulables avec les méthodes HTTP standard.

3.1.1 Caractéristiques d'une API REST

- **Sans état (Stateless)** : Chaque requête contient toutes les informations nécessaires
- **Ressources identifiées par URLs** : `/api/users/123`
- **Utilisation des méthodes HTTP** : GET, POST, PUT, DELETE
- **Format de données standardisé** : Généralement JSON
- **Codes de statut significatifs** : 200, 201, 400, 401, 404, etc.

3.1.2 Structure Typique des URLs

<code>https://api.exemple.com/v1/ressources</code>	# Liste des ressources
<code>https://api.exemple.com/v1/ressources/123</code>	# Ressource spécifique
<code>https://api.exemple.com/v1/ressources?page=2</code>	# Avec paramètres

3.2 Authentification

La plupart des APIs requièrent une authentification pour contrôler l'accès.

3.2.1 Types d'Authentification

Clé API (API Key)

```
1 import requests
2
3 API_KEY = "votre_cle_api_secrete"
4
5 # Dans les headers
6 headers = {"X-API-Key": API_KEY}
7 response = requests.get("https://api.exemple.com/data", headers=
    headers)
8
9 # Ou dans les parametres
10 params = {"api_key": API_KEY}
11 response = requests.get("https://api.exemple.com/data", params=params)
```

Bearer Token (OAuth 2.0)

```
1 import requests
2
3 ACCESS_TOKEN = "votre_token_oauth"
4
5 headers = {
6     "Authorization": f"Bearer {ACCESS_TOKEN}",
7     "Content-Type": "application/json"
8 }
9
10 response = requests.get(
11     "https://api.exemple.com/protected/resource",
12     headers=headers
13 )
```

Authentification Basique

```
1 import requests
2 from requests.auth import HTTPBasicAuth
3
4 response = requests.get(
5     "https://api.exemple.com/data",
6     auth=HTTPBasicAuth("username", "password")
7 )
8
9 # Ou plus simplement
10 response = requests.get(
11     "https://api.exemple.com/data",
12     auth=("username", "password")
13 )
```

3.3 Implémentation Complète d'un Client API

```
1 import requests
2 import time
3 from typing import Optional, Dict, Any, List
4 from dataclasses import dataclass
5 from functools import wraps
```

```
6
7 @dataclass
8 class APIConfig:
9     """Configuration pour le client API."""
10     base_url: str
11     api_key: str
12     timeout: int = 30
13     max_retries: int = 3
14     retry_delay: float = 1.0
15
16
17 def rate_limit(calls_per_minute: int):
18     """
19     Decorateur pour limiter le nombre d'appels par minute.
20     """
21     min_interval = 60.0 / calls_per_minute
22     last_call = [0.0]
23
24     def decorator(func):
25         @wraps(func)
26         def wrapper(*args, **kwargs):
27             elapsed = time.time() - last_call[0]
28             if elapsed < min_interval:
29                 time.sleep(min_interval - elapsed)
30             last_call[0] = time.time()
31             return func(*args, **kwargs)
32         return wrapper
33     return decorator
34
35
36 class APIClient:
37     """
38     Client generique pour interagir avec une API REST.
39     """
40
41     def __init__(self, config: APIConfig):
42         self.config = config
43         self.session = requests.Session()
44         self.session.headers.update({
45             "Authorization": f"Bearer {config.api_key}",
46             "Content-Type": "application/json",
47             "Accept": "application/json"
48         })
49
50     def _make_request(
51         self,
52         method: str,
53         endpoint: str,
54         params: Optional[Dict] = None,
55         data: Optional[Dict] = None
56     ) -> requests.Response:
57         """
58         Effectue une requete avec gestion des erreurs et retry.
59         """
60         url = f"{self.config.base_url}{endpoint}"
61
62         for attempt in range(self.config.max_retries):
63             try:
```

```

64         response = self.session.request(
65             method=method,
66             url=url,
67             params=params,
68             json=data,
69             timeout=self.config.timeout
70         )
71
72         # Gerer le rate limiting (429)
73         if response.status_code == 429:
74             retry_after = int(response.headers.get("Retry-
75                 After", 60))
76             print(f"Rate limit atteint. Attente de {
77                 retry_after}s...")
78             time.sleep(retry_after)
79             continue
80
81         response.raise_for_status()
82         return response
83
84     except requests.exceptions.RequestException as e:
85         if attempt < self.config.max_retries - 1:
86             delay = self.config.retry_delay * (2 ** attempt)
87             print(f"Erreur, nouvelle tentative dans {delay}s:
88                 {e}")
89             time.sleep(delay)
90         else:
91             raise
92
93 @rate_limit(calls_per_minute=60)
94 def get(self, endpoint: str, params: Optional[Dict] = None) ->
95     Dict:
96     """Requete GET."""
97     response = self._make_request("GET", endpoint, params=params)
98     return response.json()
99
100 @rate_limit(calls_per_minute=60)
101 def post(self, endpoint: str, data: Dict) -> Dict:
102     """Requete POST."""
103     response = self._make_request("POST", endpoint, data=data)
104     return response.json()
105
106 def get_paginated(
107     self,
108     endpoint: str,
109     params: Optional[Dict] = None,
110     limit: int = 100,
111     max_pages: Optional[int] = None
112 ) -> List[Dict]:
113     """
114     Recupere toutes les pages d'une ressource paginee.
115     """
116     all_items = []
117     params = params or {}
118     params["limit"] = limit
119     offset = 0
120     page = 0

```

```

118         while True:
119             params["offset"] = offset
120
121             try:
122                 data = self.get(endpoint, params)
123             except requests.exceptions.HTTPError:
124                 break
125
126             items = data.get("items", data.get("data", data.get("
127                 results", [])))
128
129             if not items:
130                 break
131
132             all_items.extend(items)
133             offset += limit
134             page += 1
135
136             print(f"Page {page}: {len(items)} elements (total: {len(
137                 all_items)}")
138
139             # Verifier s'il y a une page suivante
140             if len(items) < limit:
141                 break
142
143             if max_pages and page >= max_pages:
144                 print(f"Limite de {max_pages} pages atteinte")
145                 break
146
147             return all_items
148
149         def close(self):
150             """Ferme la session."""
151             self.session.close()
152
153         def __enter__(self):
154             return self
155
156         def __exit__(self, *args):
157             self.close()
158
159         # Exemple d'utilisation
160         if __name__ == "__main__":
161             config = APIConfig(
162                 base_url="https://api.exemple.com/v1",
163                 api_key="votre_cle_api"
164             )
165
166             with APIClient(config) as client:
167                 # Requete simple
168                 user = client.get("/users/123")
169                 print(f"Utilisateur: {user['name']}")
170
171                 # Requete avec parametres
172                 articles = client.get("/articles", params={
173                     "category": "technologie",
174                     "sort": "date",

```

```

174         "order": "desc"
175     })
176
177     # Recuperation paginee complete
178     tous_les_articles = client.get_paginated(
179         "/articles",
180         params={"category": "science"},
181         limit=100,
182         max_pages=10
183     )
184     print(f"Total articles recuperes: {len(tous_les_articles)}")

```

3.4 Exemples d'APIs Populaires

3.4.1 API OpenWeatherMap

```

1  import requests
2
3  def get_meteo(ville: str, api_key: str) -> dict:
4      """
5      Recuperer les donnees meteo pour une ville.
6      """
7      url = "https://api.openweathermap.org/data/2.5/weather"
8      params = {
9          "q": ville,
10         "appid": api_key,
11         "units": "metric",
12         "lang": "fr"
13     }
14
15     response = requests.get(url, params=params)
16     response.raise_for_status()
17     data = response.json()
18
19     return {
20         "ville": data["name"],
21         "temperature": data["main"]["temp"],
22         "description": data["weather"][0]["description"],
23         "humidite": data["main"]["humidity"],
24         "vent": data["wind"]["speed"]
25     }
26
27 # Utilisation
28 meteo = get_meteo("Paris", "VOTRE_CLE_API")
29 print(f"{meteo['ville']}: {meteo['temperature']}C, {meteo['description']}")

```

3.4.2 API GitHub

```

1  import requests
2  from typing import List, Dict
3
4  class GitHubAPI:
5      """Client pour l'API GitHub."""
6

```

```

7     def __init__(self, token: str = None):
8         self.base_url = "https://api.github.com"
9         self.headers = {"Accept": "application/vnd.github.v3+json"}
10        if token:
11            self.headers["Authorization"] = f"token {token}"
12
13        def get_user(self, username: str) -> Dict:
14            """Recupere les infos d'un utilisateur."""
15            response = requests.get(
16                f"{self.base_url}/users/{username}",
17                headers=self.headers
18            )
19            response.raise_for_status()
20            return response.json()
21
22        def get_repos(self, username: str) -> List[Dict]:
23            """Recupere les repositories d'un utilisateur."""
24            repos = []
25            page = 1
26
27            while True:
28                response = requests.get(
29                    f"{self.base_url}/users/{username}/repos",
30                    headers=self.headers,
31                    params={"page": page, "per_page": 100}
32                )
33                response.raise_for_status()
34                data = response.json()
35
36                if not data:
37                    break
38
39                repos.extend(data)
40                page += 1
41
42            return repos
43
44        def search_repos(self, query: str, language: str = None) -> List[
45            Dict]:
46            """Recherche des repositories."""
47            q = query
48            if language:
49                q += f" language:{language}"
50
51            response = requests.get(
52                f"{self.base_url}/search/repositories",
53                headers=self.headers,
54                params={"q": q, "sort": "stars", "order": "desc"}
55            )
56            response.raise_for_status()
57            return response.json()["items"]
58
59        # Utilisation
60        gh = GitHubAPI(token="ghp_votre_token") # Token optionnel
61
62        user = gh.get_user("torvalds")
63        print(f"Nom: {user['name']}, Followers: {user['followers']}")

```

```
64
65 repos = gh.get_repos("torvalds")
66 print(f"Nombre de repos: {len(repos)}")
67
68 top_python = gh.search_repos("machine learning", language="python")
69             [:5]
70 for repo in top_python:
71     print(f"- {repo['full_name']}: {repo['stargazers_count']} stars")
```

Licence IRM

Chapter 4

APIs GraphQL



Figure 4.1: Écosystème GraphQL

4.1 Introduction à GraphQL

GraphQL est un langage de requête pour APIs développé par Facebook. Contrairement à REST où le serveur définit la structure des réponses, GraphQL permet au client de spécifier exactement les données dont il a besoin.

4.1.1 Avantages par rapport à REST

- **Pas de sur-fetching** : Récupérer uniquement les champs nécessaires
- **Pas de sous-fetching** : Obtenir toutes les données liées en une seule requête
- **Un seul endpoint** : Toutes les requêtes passent par `/graphql`
- **Typage fort** : Schéma défini côté serveur avec validation
- **Introspection** : Possibilité de découvrir le schéma de l'API

4.1.2 Types de Requêtes

- **Query** : Lecture de données (équivalent GET)
- **Mutation** : Modification de données (équivalent POST/PUT/DELETE)
- **Subscription** : Flux de données en temps réel

4.2 Syntaxe des Requêtes GraphQL

4.2.1 Requête Simple

```
1 # Requete GraphQL
2 {
3   user(id: "123") {
4     id
5     name
6     email
7   }
8 }
9
10 # Reponse JSON
11 {
12   "data": {
13     "user": {
14       "id": "123",
15       "name": "Jean Dupont",
16       "email": "jean@exemple.com"
17     }
18   }
19 }
```

4.2.2 Requête avec Relations

```
1 {
2   user(id: "123") {
3     name
4     posts(first: 5) {
5       title
6       createdAt
7       comments {
8         text
9         author {
10           name
11         }
12       }
13     }
14   }
15 }
```

4.2.3 Requête avec Variables

```
1 query GetUser($userId: ID!, $postsLimit: Int) {
2   user(id: $userId) {
3     name
4     posts(first: $postsLimit) {
5       title
6     }
7   }
8 }
9
10 # Variables
11 {
12   "userId": "123",
13   "postsLimit": 10
14 }
```

4.3 Implémentation Python

4.3.1 Client GraphQL Simple

```

1 import requests
2 from typing import Dict, Any, Optional
3
4 class GraphQLClient:
5     """
6     Client generique pour les APIs GraphQL.
7     """
8
9     def __init__(self, endpoint: str, headers: Optional[Dict] = None):
10         self.endpoint = endpoint
11         self.headers = headers or {}
12         self.headers.setdefault("Content-Type", "application/json")
13
14     def execute(
15         self,
16         query: str,
17         variables: Optional[Dict] = None,
18         operation_name: Optional[str] = None
19     ) -> Dict[str, Any]:
20         """
21         Execute une requete GraphQL.
22
23         Args:
24             query: La requete GraphQL
25             variables: Variables de la requete
26             operation_name: Nom de l'operation (optionnel)
27
28         Returns:
29             Donnees de la reponse
30         """
31         payload = {"query": query}
32
33         if variables:
34             payload["variables"] = variables
35         if operation_name:
36             payload["operationName"] = operation_name
37
38         response = requests.post(
39             self.endpoint,
40             json=payload,
41             headers=self.headers,
42             timeout=30
43         )
44         response.raise_for_status()
45
46         result = response.json()
47
48         # Verifier les erreurs GraphQL
49         if "errors" in result:
50             errors = result["errors"]
51             error_messages = [e.get("message", str(e)) for e in errors]
52             raise Exception(f"Erreurs GraphQL: {error_messages}")
53

```

```

54         return result.get("data", {})
55
56
57 # Exemple d'utilisation
58 client = GraphQLClient(
59     endpoint="https://api.exemple.com/graphql",
60     headers={"Authorization": "Bearer TOKEN"}
61 )
62
63 # Requete simple
64 query = """
65 {
66     users(first: 10) {
67         id
68         name
69         email
70     }
71 }
72 """
73 data = client.execute(query)
74 for user in data["users"]:
75     print(f"- {user['name']} ({user['email']})")

```

4.3.2 Requêtes avec Variables

```

1  import requests
2
3  def rechercher_repositories(
4      client: GraphQLClient,
5      terme: str,
6      langage: str,
7      limite: int = 10
8  ) -> list:
9      """
10     Recherche des repositories GitHub via GraphQL.
11     """
12     query = """
13     query SearchRepos($queryString: String!, $first: Int!) {
14         search(query: $queryString, type: REPOSITORY, first: $first) {
15             repositoryCount
16             edges {
17                 node {
18                     ... on Repository {
19                         name
20                         description
21                         url
22                         stargazerCount
23                         forkCount
24                         primaryLanguage {
25                             name
26                         }
27                         owner {
28                             login
29                         }
30                     }
31                 }
32             }
33         }
34     }
35     """

```

```

33     }
34 }
35 """
36
37 variables = {
38     "queryString": f"{terme} language:{langage}",
39     "first": limite
40 }
41
42 data = client.execute(query, variables)
43
44 repositories = []
45 for edge in data["search"]["edges"]:
46     repo = edge["node"]
47     repositories.append({
48         "nom": repo["name"],
49         "description": repo["description"],
50         "url": repo["url"],
51         "stars": repo["stargazerCount"],
52         "forks": repo["forkCount"],
53         "langage": repo["primaryLanguage"]["name"] if repo["
54             primaryLanguage"] else None,
55         "owner": repo["owner"]["login"]
56     })
57
58 return repositories
59
60 # Utilisation avec l'API GitHub GraphQL
61 github_client = GraphQLClient(
62     endpoint="https://api.github.com/graphql",
63     headers={"Authorization": "Bearer ghp_votre_token"}
64 )
65
66 repos = rechercher_repositories(
67     github_client,
68     terme="machine learning",
69     langage="python",
70     limite=5
71 )
72
73 print(f"Trouve {len(repos)} repositories:")
74 for repo in repos:
75     print(f"    - {repo['owner']}/{repo['nom']}: {repo['stars']} stars")

```

4.3.3 Mutations GraphQL

```

1 def creer_issue_github(
2     client: GraphQLClient,
3     repo_id: str,
4     titre: str,
5     description: str
6 ) -> dict:
7     """
8     Cree une issue sur un repository GitHub.
9     """
10    mutation = """

```

```

11     mutation CreateIssue($input: CreateIssueInput!) {
12         createIssue(input: $input) {
13             issue {
14                 id
15                 number
16                 title
17                 url
18                 createdAt
19             }
20         }
21     }
22     """
23
24     variables = {
25         "input": {
26             "repositoryId": repo_id,
27             "title": titre,
28             "body": description
29         }
30     }
31
32     data = client.execute(mutation, variables)
33     return data["createIssue"]["issue"]
34
35
36 # Utilisation
37 nouvelle_issue = creer_issue_github(
38     github_client,
39     repo_id="R_kgD0xxxxxx",
40     titre="Bug: Probleme de performance",
41     description="Description detaillee du probleme..."
42 )
43 print(f"Issue creee: #{nouvelle_issue['number']} - {nouvelle_issue['url']}")

```

4.4 Introspection du Schéma

```

1 def obtenir_schema(client: GraphQLClient) -> dict:
2     """
3     Recupere le schema complet de l'API GraphQL.
4     """
5     query = """
6     {
7         __schema {
8             types {
9                 name
10                description
11                fields {
12                    name
13                    type {
14                        name
15                        kind
16                    }
17                }
18            }
19            queryType {

```

```
20         name
21     }
22     mutationType {
23         name
24     }
25     }
26 }
27 """
28
29 return client.execute(query)
30
31
32 def lister_types_disponibles(client: GraphQLClient) -> list:
33     """
34     Liste tous les types disponibles dans l'API.
35     """
36     schema = obtenir_schema(client)
37
38     types_utilisateur = []
39     for type_info in schema["__schema"]["types"]:
40         # Ignorer les types internes (commencent par __)
41         if not type_info["name"].startswith("__"):
42             types_utilisateur.append({
43                 "nom": type_info["name"],
44                 "description": type_info["description"],
45                 "champs": [f["name"] for f in (type_info["fields"] or
46                                         [])]
47             })
48
49     return types_utilisateur
```

Chapter 5

Web Scraping

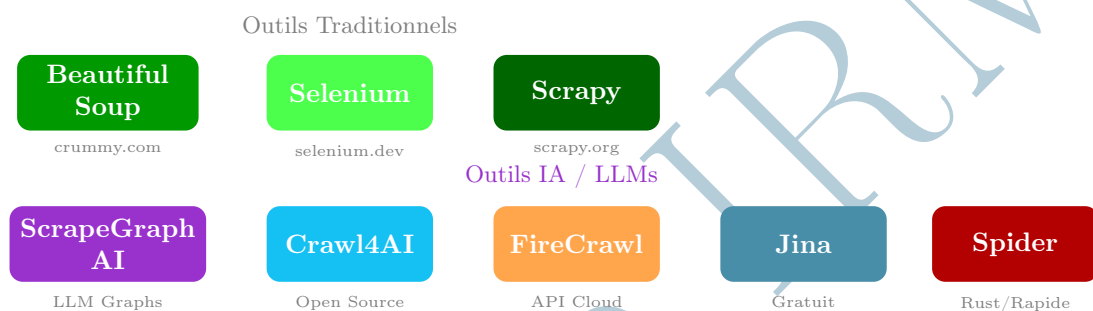


Figure 5.1: Outils de Web Scraping Python (Traditionnels + IA)

5.1 Introduction au Web Scraping

Le web scraping est une technique d'extraction automatisée de données depuis des pages web. Il est utilisé lorsqu'aucune API n'est disponible pour accéder aux données souhaitées.

5.1.1 Cas d'Utilisation Légitimes

- Agrégation de prix pour comparaison
- Surveillance de sites web (changements de contenu)
- Collecte de données publiques pour la recherche
- Archivage de contenu web
- Alimentation de moteurs de recherche

5.1.2 Considérations Éthiques et Légales

- **Respecter le fichier robots.txt** : Indique les pages autorisées au scraping
- **Respecter les conditions d'utilisation** du site
- **Ne pas surcharger les serveurs** : Espacer les requêtes
- **Identifier votre bot** : User-Agent transparent
- **Respecter les données personnelles** : RGPD et autres réglementations

5.2 Scraping Statique avec BeautifulSoup

Le scraping statique concerne les pages dont le contenu est présent directement dans le HTML initial, sans nécessiter l'exécution de JavaScript.

5.2.1 Installation

```
1 pip install beautifulsoup4 lxml requests
```

5.2.2 Concepts Fondamentaux

```
1 from bs4 import BeautifulSoup
2 import requests
3
4 # Telecharger la page
5 url = "https://exemple.com/page"
6 response = requests.get(url)
7 html = response.text
8
9 # Parser le HTML
10 soup = BeautifulSoup(html, "lxml") # lxml est plus rapide que
    html.parser
11
12 # Navigation dans le DOM
13 titre = soup.title.text           # Balise <title>
14 premier_h1 = soup.h1.text         # Premier <h1>
15 tous_les_liens = soup.find_all("a") # Toutes les
    balises <a>
```

5.2.3 Sélecteurs CSS

```
1 from bs4 import BeautifulSoup
2
3 html = """
4 <div class="container">
5 <article class="post" id="article-1">
6 <h2 class="titre">Premier Article</h2>
7 <p class="contenu">Contenu du premier article...</p>
8 <span class="auteur" data-id="42">Jean Dupont</span>
9 </article>
10 <article class="post" id="article-2">
11 <h2 class="titre">Deuxieme Article</h2>
12 <p class="contenu">Contenu du deuxieme article...</p>
13 <span class="auteur" data-id="17">Marie Martin</span>
14 </article>
15 </div>
16 """
17
18 soup = BeautifulSoup(html, "lxml")
19
20 # Selection par classe CSS
21 articles = soup.select("article.post")
22
23 # Selection par ID
24 article_1 = soup.select_one("#article-1")
```

```

25     # Selection imbriquee
26     titres = soup.select("article.post h2.titre")
27
28     # Selection par attribut
29     auteurs = soup.select("span[data-id]")
30
31     # Combinaison de selecteurs
32     contenu_premier = soup.select_one("#article-1 p.contenu")
33

```

5.2.4 Scraper Complet avec Gestion d'Erreurs

```

1     import requests
2     from bs4 import BeautifulSoup
3     from typing import List, Dict, Optional
4     import time
5     import random
6     from dataclasses import dataclass
7     import logging
8
9     logging.basicConfig(level=logging.INFO)
10    logger = logging.getLogger(__name__)
11
12
13    @dataclass
14    class Article:
15        """Structure pour un article scrape."""
16        titre: str
17        url: str
18        auteur: str
19        date: str
20        contenu: str
21        tags: List[str]
22
23
24    class WebScraper:
25        """
26        Scraper generique avec bonnes pratiques integrees.
27        """
28
29    def __init__(
30        self,
31        base_url: str,
32        delay_range: tuple = (1, 3),
33        max_retries: int = 3
34    ):
35        self.base_url = base_url
36        self.delay_range = delay_range
37        self.max_retries = max_retries
38        self.session = requests.Session()
39        self.session.headers.update({
40            "User-Agent": "MonScraper/1.0 (contact@exemple.com)",
41            "Accept": "text/html,application/xhtml+xml",
42            "Accept-Language": "fr-FR,fr;q=0.9",
43        })
44
45    def _respecter_delai(self):

```

```

46     """Attend un delai aleatoire entre les requetes."""
47     delai = random.uniform(*self.delay_range)
48     time.sleep(delai)
49
50     def _get_page(self, url: str) -> Optional[BeautifulSoup]:
51         """
52         Telecharge et parse une page avec retry.
53         """
54         for attempt in range(self.max_retries):
55             try:
56                 self._respecter_delai()
57                 response = self.session.get(url, timeout=30)
58                 response.raise_for_status()
59                 return BeautifulSoup(response.text, "lxml")
60
61             except requests.exceptions.RequestException as e:
62                 logger.warning(f"Tentative {attempt + 1}/{self.max_retries} "
63                                f"echouee pour {url}: {e}")
64                 if attempt < self.max_retries - 1:
65                     time.sleep(2 ** attempt) # Backoff exponentiel
66                 else:
67                     logger.error(f"Echec definitif pour {url}")
68                 return None
69
70     def extraire_article(self, url: str) -> Optional[Article]:
71         """
72         Extrait les donnees d'un article.
73         """
74         soup = self._get_page(url)
75         if not soup:
76             return None
77
78         try:
79             # Adapter ces selecteurs au site cible
80             titre = soup.select_one("h1.article-title")
81             auteur = soup.select_one("span.author-name")
82             date = soup.select_one("time.publish-date")
83             contenu = soup.select_one("div.article-content")
84             tags = soup.select("a.tag")
85
86             return Article(
87                 titre=titre.text.strip() if titre else "",
88                 url=url,
89                 auteur=auteur.text.strip() if auteur else "Inconnu",
90                 date=date.get("datetime", "") if date else "",
91                 contenu=contenu.text.strip() if contenu else "",
92                 tags=[t.text.strip() for t in tags]
93             )
94
95         except Exception as e:
96             logger.error(f"Erreur extraction {url}: {e}")
97             return None
98
99     def extraire_liste_articles(self, url_liste: str) -> List[str]
100     ]:
101         """
102         Extrait les URLs des articles depuis une page liste.
103         """

```

```

103     soup = self._get_page(url_liste)
104     if not soup:
105         return []
106
107     # Adapter ce selecteur au site cible
108     liens = soup.select("article.post a.article-link")
109
110     urls = []
111     for lien in liens:
112         href = lien.get("href", "")
113         if href.startswith("/"):
114             href = self.base_url + href
115         urls.append(href)
116
117     return urls
118
119     def scraper_site(
120     self,
121     url_depart: str,
122     max_articles: int = 100
123     ) -> List[Article]:
124         """
125         Scrape un site en suivant la pagination.
126         """
127         articles = []
128         page = 1
129
130         while len(articles) < max_articles:
131             url_page = f"{url_depart}?page={page}"
132             logger.info(f"Scraping page {page}...")
133
134             urls_articles = self.extraire_liste_articles(url_page)
135
136             if not urls_articles:
137                 logger.info("Plus d'articles trouves")
138                 break
139
140             for url in urls_articles:
141                 if len(articles) >= max_articles:
142                     break
143
144             article = self.extraire_article(url)
145             if article:
146                 articles.append(article)
147                 logger.info(f"Article extrait: {article.titre[:50]}...")
148
149             page += 1
150
151         return articles
152
153     def close(self):
154         """Ferme la session."""
155         self.session.close()
156
157
158     # Utilisation
159     if __name__ == "__main__":
160         scraper = WebScraper(

```

```

161     base_url="https://blog.exemple.com",
162     delay_range=(2, 5)
163 )
164
165     try:
166         articles = scraper.scrap_site(
167             "https://blog.exemple.com/articles",
168             max_articles=50
169         )
170
171         print(f"\n{len(articles)} articles extraits:")
172         for art in articles[:5]:
173             print(f"    - {art.titre} par {art.auteur}")
174
175     finally:
176         scraper.close()

```

5.3 Scraping Dynamique avec Selenium

Selenium est nécessaire lorsque le contenu est généré par JavaScript après le chargement initial de la page.

5.3.1 Installation

```

1 pip install selenium webdriver-manager

```

5.3.2 Configuration du Driver

```

1 from selenium import webdriver
2 from selenium.webdriver.chrome.service import Service
3 from selenium.webdriver.chrome.options import Options
4 from webdriver_manager.chrome import ChromeDriverManager
5
6 def creer_driver(headless: bool = True) -> webdriver.Chrome:
7     """
8     Cree et configure un driver Chrome.
9     """
10    options = Options()
11
12    if headless:
13        options.add_argument("--headless") # Mode sans interface
14
15    # Options recommandees
16    options.add_argument("--no-sandbox")
17    options.add_argument("--disable-dev-shm-usage")
18    options.add_argument("--disable-gpu")
19    options.add_argument("--window-size=1920,1080")
20
21    # User-Agent personnalise
22    options.add_argument(
23        "user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
24        "AppleWebKit/537.36 Chrome/91.0.4472.124 Safari/537.36"
25    )
26

```

```
27     # Installation automatique du driver
28     service = Service(ChromeDriverManager().install())
29
30     return webdriver.Chrome(service=service, options=options)
```

5.3.3 Attente des Éléments

```
1     from selenium.webdriver.common.by import By
2     from selenium.webdriver.support.ui import WebDriverWait
3     from selenium.webdriver.support import expected_conditions as
4         EC
5     from selenium.common.exceptions import TimeoutException
6
7     def attendre_element(driver, selecteur_css: str, timeout: int
8         = 10):
9         """
10        Attend qu'un element soit present dans le DOM.
11        """
12        try:
13            element = WebDriverWait(driver, timeout).until(
14                EC.presence_of_element_located((By.CSS_SELECTOR, selecteur_css
15                )))
16        )
17        return element
18    except TimeoutException:
19        print(f"Timeout: element '{selecteur_css}' non trouve")
20        return None
21
22    def attendre_elements(driver, selecteur_css: str, timeout: int
23        = 10):
24        """
25        Attend que plusieurs elements soient presents.
26        """
27        try:
28            elements = WebDriverWait(driver, timeout).until(
29                EC.presence_of_all_elements_located((By.CSS_SELECTOR,
30                selecteur_css)))
31        )
32        return elements
33    except TimeoutException:
34        return []
35
36    def attendre_cliquable(driver, selecteur_css: str, timeout:
37        int = 10):
38        """
39        Attend qu'un element soit cliquable.
40        """
41        try:
42            element = WebDriverWait(driver, timeout).until(
43                EC.element_to_be_clickable((By.CSS_SELECTOR, selecteur_css)))
44        )
45        return element
46    except TimeoutException:
47        return None
```

5.3.4 Scraper Dynamique Complet

```

1  from selenium import webdriver
2  from selenium.webdriver.common.by import By
3  from selenium.webdriver.support.ui import WebDriverWait
4  from selenium.webdriver.support import expected_conditions as
    EC
5  from selenium.webdriver.common.keys import Keys
6  from selenium.common.exceptions import TimeoutException,
    NoSuchElementException
7  from bs4 import BeautifulSoup
8  import time
9  from typing import List, Dict
10
11
12  class SeleniumScraper:
13      """
14      Scraper pour sites dynamiques avec Selenium.
15      """
16
17      def __init__(self, headless: bool = True):
18          self.driver = creer_driver(headless)
19          self.wait = WebDriverWait(self.driver, 10)
20
21      def naviguer(self, url: str):
22          """Navigue vers une URL."""
23          self.driver.get(url)
24          time.sleep(2) # Attendre le chargement initial
25
26      def scroll_infini(self, max_scrolls: int = 10) -> int:
27          """
28          Gere le scroll infini pour charger plus de contenu.
29          Retourne le nombre d'elements charges.
30          """
31          dernier_hauteur = self.driver.execute_script(
32              "return document.body.scrollHeight"
33          )
34
35          for i in range(max_scrolls):
36              # Scroller vers le bas
37              self.driver.execute_script(
38                  "window.scrollTo(0, document.body.scrollHeight);"
39              )
40              time.sleep(2) # Attendre le chargement
41
42              # Verifier si nouveau contenu charge
43              nouveau_hauteur = self.driver.execute_script(
44                  "return document.body.scrollHeight"
45              )
46
47              if nouveau_hauteur == dernier_hauteur:
48                  print(f"Fin du scroll apres {i + 1} iterations")
49                  break
50
51              dernier_hauteur = nouveau_hauteur
52
53          return i + 1
54

```

```

55     def cliquer_bouton(self, selecteur: str) -> bool:
56         """Clique sur un bouton."""
57         try:
58             bouton = self.wait.until(
59                 EC.element_to_be_clickable((By.CSS_SELECTOR, selecteur))
60             )
61             bouton.click()
62             time.sleep(1)
63             return True
64         except TimeoutException:
65             return False
66
67     def remplir_champ(self, selecteur: str, texte: str):
68         """Remplit un champ de formulaire."""
69         champ = self.wait.until(
70             EC.presence_of_element_located((By.CSS_SELECTOR, selecteur))
71         )
72         champ.clear()
73         champ.send_keys(texte)
74
75     def soumettre_formulaire(self, selecteur_form: str):
76         """Soumet un formulaire."""
77         form = self.driver.find_element(By.CSS_SELECTOR,
78                                         selecteur_form)
79         form.submit()
80         time.sleep(2)
81
82     def obtenir_html(self) -> str:
83         """Retourne le HTML actuel de la page."""
84         return self.driver.page_source
85
86     def obtenir_soup(self) -> BeautifulSoup:
87         """Retourne un objet BeautifulSoup du HTML actuel."""
88         return BeautifulSoup(self.obtenir_html(), "lxml")
89
90     def extraire_produits_ecommerce(self, url: str) -> List[Dict]:
91         """
92         Exemple: extrait les produits d'un site e-commerce dynamique.
93         """
94         self.naviguer(url)
95
96         # Charger tout le contenu via scroll
97         self.scroll_infini(max_scrolls=5)
98
99         # Fermer les popups eventuels
100        try:
101            self.cliquer_bouton("button.close-popup")
102        except:
103            pass
104
105        # Parser le contenu charge
106        soup = self.obtenir_soup()
107        produits = []
108
109        for item in soup.select("div.product-item"):
110            try:
111                produit = {
112                    "nom": item.select_one(".product-name").text.strip(),

```

```

112         "prix": item.select_one(".product-price").text.strip()
113         ,
114         "url": item.select_one("a.product-link")["href"],
115         "image": item.select_one("img.product-image")["src"],
116         "note": item.select_one(".rating").text.strip()
117         if item.select_one(".rating") else None,
118     }
119     produits.append(produit)
120 except (AttributeError, KeyError) as e:
121     continue
122
123 return produits
124
125 def rechercher_et_extraire(
126     self,
127     url: str,
128     terme_recherche: str,
129     selecteur_recherche: str,
130     selecteur_resultats: str
131 ) -> List[Dict]:
132     """
133     Effectue une recherche et extrait les resultats.
134     """
135     self.naviguer(url)
136
137     # Remplir le champ de recherche
138     self.remplir_champ(selecteur_recherche, terme_recherche)
139
140     # Soumettre (Enter)
141     champ = self.driver.find_element(By.CSS_SELECTOR,
142                                     selecteur_recherche)
143     champ.send_keys(Keys.RETURN)
144
145     # Attendre les resultats
146     time.sleep(3)
147
148     # Extraire
149     soup = self.obtenir_soup()
150     resultats = []
151
152     for item in soup.select(selecteur_resultats):
153         resultats.append({
154             "titre": item.select_one("h2, h3, .title").text.strip()
155             ()
156             if item.select_one("h2, h3, .title") else "",
157             "lien": item.select_one("a")["href"]
158             if item.select_one("a") else "",
159         })
160
161     return resultats
162
163 def capturer_screenshot(self, chemin: str):
164     """Capture une capture d'ecran."""
165     self.driver.save_screenshot(chemin)
166
167 def close(self):
168     """Ferme le navigateur."""
169     self.driver.quit()

```

```

167
168     def __enter__(self):
169         return self
170
171     def __exit__(self, *args):
172         self.close()
173
174
175     # Utilisation
176     if __name__ == "__main__":
177         with SeleniumScraper(headless=True) as scraper:
178             # Exemple e-commerce
179             produits = scraper.extraire_produits_ecommerce(
180                 "https://boutique.exemple.com/categorie/electronique"
181             )
182
183             print(f"{len(produits)} produits trouves:")
184             for p in produits[:5]:
185                 print(f"    - {p['nom']}: {p['prix']}")

```

5.4 Scraping à Grande Échelle avec Scrapy

Scrapy est un framework complet pour le scraping à grande échelle avec gestion automatique de la concurrence, des erreurs et de la persistance.

5.4.1 Installation

```

1 pip install scrapy

```

5.4.2 Structure d'un Projet Scrapy

```

mon_projet/
scrapy.cfg
mon_projet/
__init__.py
items.py
middlewares.py
pipelines.py
settings.py
spiders/
__init__.py
mon_spider.py

```

5.4.3 Définition des Items

```

1 # items.py
2 import scrapy
3
4 class ArticleItem(scrapy.Item):
5     """Structure pour un article."""
6     titre = scrapy.Field()
7     url = scrapy.Field()
8     auteur = scrapy.Field()

```

```
9     date = scrapy.Field()
10     contenu = scrapy.Field()
11     tags = scrapy.Field()
12     source = scrapy.Field()
```

5.4.4 Spider Complet

```
1     # spiders/articles_spider.py
2     import scrapy
3     from mon_projet.items import ArticleItem
4     from urllib.parse import urljoin
5
6
7     class ArticlesSpider(scrapy.Spider):
8         """
9         Spider pour extraire des articles d'un blog.
10        """
11        name = "articles"
12        allowed_domains = ["blog.exemple.com"]
13        start_urls = ["https://blog.exemple.com/articles"]
14
15        # Configuration spécifique au spider
16        custom_settings = {
17            "DOWNLOAD_DELAY": 2,
18            "CONCURRENT_REQUESTS_PER_DOMAIN": 2,
19        }
20
21        def parse(self, response):
22            """
23            Parse la page liste et suit les liens des articles.
24            """
25            # Extraire les liens des articles
26            for article_link in response.css("article.post a.article-link"):
27                url = article_link.attrib["href"]
28                yield response.follow(url, callback=self.parse_article)
29
30            # Suivre la pagination
31            next_page = response.css("a.next-page::attr(href)").get()
32            if next_page:
33                yield response.follow(next_page, callback=self.parse)
34
35        def parse_article(self, response):
36            """
37            Parse une page article et extrait les donnees.
38            """
39            item = ArticleItem()
40
41            item["titre"] = response.css("h1.article-title::text").get()
42            item["url"] = response.url
43            item["auteur"] = response.css("span.author-name::text").get()
44            item["date"] = response.css("time.publish-date::attr(datetime)").get()
45
46            # Extraire le contenu (plusieurs paragraphes)
47            paragraphes = response.css("div.article-content p::text").getall()
```

```
48     item["contenu"] = " ".join(paragraphes)
49
50     # Extraire les tags
51     item["tags"] = response.css("a.tag::text").getall()
52
53     item["source"] = self.name
54
55     yield item
```

5.4.5 Configuration (settings.py)

```
1     # settings.py
2
3     # Identification du bot
4     BOT_NAME = "mon_projet"
5     USER_AGENT = "MonScraper/1.0 (+https://exemple.com/bot)"
6
7     # Respect du robots.txt
8     ROBOTSTXT_OBEY = True
9
10    # Delais et concurrence
11    DOWNLOAD_DELAY = 1
12    CONCURRENT_REQUESTS = 16
13    CONCURRENT_REQUESTS_PER_DOMAIN = 4
14
15    # Retry
16    RETRY_TIMES = 3
17    RETRY_HTTP_CODES = [500, 502, 503, 504, 408, 429]
18
19    # Cache HTTP (developpement)
20    HTTPCACHE_ENABLED = True
21    HTTPCACHE_EXPIRATION_SECS = 86400
22
23    # Pipeline de traitement
24    ITEM_PIPELINES = {
25        "mon_projet.pipelines.NettoyagePipeline": 100,
26        "mon_projet.pipelines.ValidationPipeline": 200,
27        "mon_projet.pipelines.StockagePipeline": 300,
28    }
29
30    # Export automatique
31    FEEDS = {
32        "output/articles.json": {
33            "format": "json",
34            "encoding": "utf-8",
35            "indent": 2,
36        },
37        "output/articles.csv": {
38            "format": "csv",
39            "encoding": "utf-8",
40        },
41    }
```

5.4.6 Pipelines de Traitement

```

1  # pipelines.py
2  from itemadapter import ItemAdapter
3  import re
4
5
6  class NettoyagePipeline:
7      """Nettoie les donnees extraites."""
8
9      def process_item(self, item, spider):
10         adapter = ItemAdapter(item)
11
12         # Nettoyer le titre
13         if adapter.get("titre"):
14             adapter["titre"] = adapter["titre"].strip()
15
16         # Nettoyer le contenu
17         if adapter.get("contenu"):
18             contenu = adapter["contenu"]
19             contenu = re.sub(r"\s+", " ", contenu) # Espaces multiples
20             contenu = contenu.strip()
21             adapter["contenu"] = contenu
22
23         # Nettoyer les tags
24         if adapter.get("tags"):
25             adapter["tags"] = [t.strip().lower() for t in adapter["tags"]]
26
27         return item
28
29
30     class ValidationPipeline:
31         """Valide les donnees et rejette les items incomplets."""
32
33         def process_item(self, item, spider):
34             adapter = ItemAdapter(item)
35
36             # Verifier les champs obligatoires
37             if not adapter.get("titre"):
38                 raise DropItem(f"Article sans titre: {item}")
39
40             if not adapter.get("contenu") or len(adapter["contenu"]) <
41                 100:
42                 raise DropItem(f"Contenu trop court: {item}")
43
44             return item
45
46
47     class StockagePipeline:
48         """Stocke les items dans une base de donnees."""
49
50         def open_spider(self, spider):
51             # Initialiser la connexion DB
52             import sqlite3
53             self.conn = sqlite3.connect("articles.db")
54             self.cursor = self.conn.cursor()
55             self.cursor.execute("""
56                 CREATE TABLE IF NOT EXISTS articles (
57                     id INTEGER PRIMARY KEY,
58                     titre TEXT,

```

```

58         url TEXT UNIQUE,
59         auteur TEXT,
60         date TEXT,
61         contenu TEXT,
62         tags TEXT
63     )
64     """
65
66     def close_spider(self, spider):
67         self.conn.commit()
68         self.conn.close()
69
70     def process_item(self, item, spider):
71         adapter = ItemAdapter(item)
72
73         try:
74             self.cursor.execute("""
75             INSERT OR REPLACE INTO articles
76             (titre, url, auteur, date, contenu, tags)
77             VALUES (?, ?, ?, ?, ?, ?)
78             """, (
79                 adapter["titre"],
80                 adapter["url"],
81                 adapter.get("auteur"),
82                 adapter.get("date"),
83                 adapter.get("contenu"),
84                 ",".join(adapter.get("tags", [])),
85             ))
86         except Exception as e:
87             spider.logger.error(f"Erreur DB: {e}")
88
89         return item

```

5.4.7 Exécution du Spider

```

1     # Ligne de commande
2     scrapy crawl articles
3
4     # Avec export spécifique
5     scrapy crawl articles -o resultats.json
6
7     # Avec arguments
8     scrapy crawl articles -a categorie=technologie -a max_pages=10

```

5.5 Scraping Intelligent avec l'IA : ScrapeGraphAI

L'émergence des grands modèles de langage (LLMs) a révolutionné le web scraping en permettant une extraction de données intelligente et adaptative. **ScrapeGraphAI** est une bibliothèque Python open-source qui exploite la puissance des LLMs pour créer des pipelines de scraping flexibles et auto-adaptatifs.

5.5.1 Principe et Avantages

Contrairement aux outils traditionnels qui reposent sur des sélecteurs CSS ou XPath fixes, ScrapeGraphAI utilise l'intelligence artificielle pour :

- **Comprendre le contenu** : Le LLM analyse sémantiquement la page web
- **S'adapter aux changements** : Plus besoin de réécrire les scripts quand le site change
- **Extraction en langage naturel** : Décrivez simplement ce que vous voulez extraire
- **Structuration automatique** : Les données sont retournées en JSON structuré



Figure 5.2: ScrapeGraphAI : Scraping propulsé par l'IA

5.5.2 Installation

```
1 pip install scrapegraphai
2 playwright install
```

Pour utiliser des modèles locaux avec Ollama :

```
1 # Installer Ollama depuis https://ollama.com/
2 # Puis télécharger un modèle
3 ollama pull llama3.2
4 ollama pull nomic-embed-text
```

5.5.3 Exemple Simple avec OpenAI

```
1 from scrapegraphai.graphs import SmartScrapperGraph
2 import json
3
4 # Configuration avec OpenAI
5 graph_config = {
6     "llm": {
7         "api_key": "VOTRE_CLE_OPENAI",
8         "model": "openai/gpt-4o-mini",
9     },
10    "verbose": True,
11    "headless": True,
12 }
13
14 # Créer le scraper intelligent
15 smart_scraper = SmartScrapperGraph(
16     prompt="Extrais le nom de l'entreprise, sa description, "
17     "et les emails de contact",
18     source="https://exemple.com/a-propos",
19     config=graph_config
20 )
21
22 # Executer le scraping
```

```

23     result = smart_scraper.run()
24
25     # Afficher le resultat structure
26     print(json.dumps(result, indent=4, ensure_ascii=False))

```

Exemple de sortie :

```

1     {
2         "nom_entreprise": "TechCorp Solutions",
3         "description": "Leader en solutions cloud pour
4             entreprises",
5         "emails": [
6             "contact@techcorp.com",
7             "support@techcorp.com"
8         ]
9     }

```

5.5.4 Utilisation avec Modèles Locaux (Ollama)

Pour une utilisation gratuite et privée, vous pouvez utiliser des modèles locaux :

```

1     from scrapegraphai.graphs import SmartScraperGraph
2     import json
3
4     # Configuration avec Ollama (modele local)
5     graph_config = {
6         "llm": {
7             "model": "ollama/llama3.2",
8             "temperature": 0,
9             "format": "json",
10            "base_url": "http://localhost:11434",
11        },
12        "embeddings": {
13            "model": "ollama/nomic-embed-text",
14            "base_url": "http://localhost:11434",
15        },
16        "verbose": True,
17        "headless": True,
18    }
19
20    # Scraper des articles de blog
21    smart_scraper = SmartScraperGraph(
22        prompt="Liste tous les articles avec leur titre, date et
23            resume",
24        source="https://blog.exemple.com/articles",
25        config=graph_config
26    )
27
28    result = smart_scraper.run()
29    print(json.dumps(result, indent=4, ensure_ascii=False))

```

5.5.5 Types de Pipelines Disponibles

ScrapeGraphAI propose plusieurs pipelines selon vos besoins :

Pipeline	Description
SmartScraperGraph	Scraping d'une seule page avec prompt utilisateur
SmartScraperMultiGraph	Scraping de plusieurs URLs simultanément
SearchGraph	Recherche sur moteur de recherche + extraction
ScriptCreatorGraph	Génère un script Python BeautifulSoup/Sele- nium
SpeechGraph	Extraction + génération audio (text-to-speech)
JSONScraperGraph	Extraction depuis fichiers JSON locaux

5.5.6 Scraping Multi-Pages

```

1  from scrapegraphai.graphs import SmartScraperMultiGraph
2
3  # Liste d'URLs a scraper
4  urls = [
5  "https://site.com/produit/1",
6  "https://site.com/produit/2",
7  "https://site.com/produit/3",
8  ]
9
10 graph_config = {
11     "llm": {
12         "model": "ollama/llama3.2",
13         "temperature": 0,
14         "format": "json",
15         "base_url": "http://localhost:11434",
16     },
17     "embeddings": {
18         "model": "ollama/nomic-embed-text",
19         "base_url": "http://localhost:11434",
20     },
21 }
22
23 # Scraper multi-pages
24 multi_scraper = SmartScraperMultiGraph(
25     prompt="Extrais le nom du produit, le prix et la disponibilité",
26     source=urls,
27     config=graph_config
28 )
29
30 resultats = multi_scraper.run()
31
32 for i, produit in enumerate(resultats):
33     print(f"Produit {i+1}: {produit}")

```

5.5.7 Génération Automatique de Scripts

ScrapeGraphAI peut générer automatiquement un script de scraping traditionnel :

```

1  from scrapegraphai.graphs import ScriptCreatorGraph

```

```

2
3     graph_config = {
4         "llm": {
5             "api_key": "VOTRE_CLE_OPENAI",
6             "model": "openai/gpt-4o-mini",
7         },
8         "library": "beautifulsoup" # ou "selenium"
9     }
10
11     # Generer un script de scraping
12     script_creator = ScriptCreatorGraph(
13         prompt="Cree un script pour extraire tous les titres "
14         "et liens des articles",
15         source="https://news.exemple.com",
16         config=graph_config
17     )
18
19     # Obtenir le script Python genere
20     script_python = script_creator.run()
21     print(script_python)
22
23     # Sauvegarder le script
24     with open("scraper_genere.py", "w") as f:
25         f.write(script_python)

```

5.5.8 Comparaison : Scraping Traditionnel vs IA

Critère	Traditionnel	ScrapeGraphAI
Sélecteurs CSS/XPath	Requis	Non requis
Adaptation aux changements	Manuelle	Automatique
Courbe d'apprentissage	Moyenne	Faible
Coût	Gratuit	API payante ou GPU local
Précision	Très haute	Haute (dépend du LLM)
Vitesse	Très rapide	Plus lente
Volume de données	Illimité	Limité par tokens

5.5.9 Bonnes Pratiques

1. **Prompts précis** : Plus votre description est claire, meilleurs sont les résultats
2. **Modèles locaux** : Utilisez Ollama pour les projets avec beaucoup de pages (économie de coûts)
3. **Validation** : Vérifiez toujours les résultats, les LLMs peuvent halluciner
4. **Combinaison** : Utilisez ScrapeGraphAI pour le prototypage, puis ScriptCreatorGraph pour générer un script traditionnel optimisé
5. **Rate limiting** : Respectez les délais entre requêtes même avec l'IA

5.5.10 Ressources

- Documentation officielle : <https://scrapegraph-doc.onrender.com>

- GitHub : <https://github.com/ScrapeGraphAI/Scrapegraph-ai>
- PyPI : `pip install scrapegraphai`

5.6 Crawl4AI : Crawler Open Source pour LLMs

Crawl4AI est un crawler web open-source (50k+ stars sur GitHub) spécialement conçu pour les applications d'IA. Il transforme les pages web en Markdown propre, idéal pour les pipelines RAG et le fine-tuning de modèles.

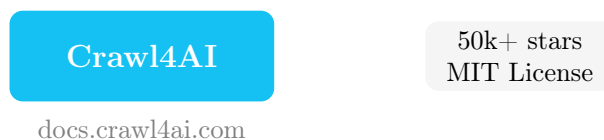


Figure 5.3: Crawl4AI : Crawler optimisé pour les LLMs

5.6.1 Caractéristiques

- **Output LLM-Ready** : Conversion automatique en Markdown propre
- **100% gratuit** : Pas de clé API obligatoire, open source
- **Architecture async** : Crawling parallèle haute performance
- **Deep Crawling** : Stratégies BFS et DFS pour explorer plusieurs niveaux
- **Extraction LLM** : Support OpenAI, Claude, Ollama (local)

5.6.2 Installation et Exemple

```

1  # Installation
2  pip install crawl4ai
3  crawl4ai-setup # Installe les navigateurs

```

```

1  import asyncio
2  from crawl4ai import AsyncWebCrawler
3
4  async def crawl_simple():
5      async with AsyncWebCrawler(verbose=True) as crawler:
6          result = await crawler.arun(url="https://exemple.com")
7
8      # Contenu en Markdown (pret pour LLM)
9      print(result.markdown)
10
11     # Metadonnees
12     print(f"Titre: {result.metadata.get('title')}")
13     print(f"Liens: {len(result.links)}")
14
15     asyncio.run(crawl_simple())

```

5.6.3 Extraction Structurée avec LLM

```

1  from crawl4ai import AsyncWebCrawler, CrawlerRunConfig
2  from crawl4ai.extraction_strategy import LLMExtractionStrategy

```

```

3     from pydantic import BaseModel, Field
4     import os
5
6     class ProduitInfo(BaseModel):
7         nom: str = Field(..., description="Nom du produit")
8         prix: str = Field(..., description="Prix")
9         disponible: bool = Field(..., description="En stock")
10
11     async def extraction_llm():
12         extraction_strategy = LLMExtractionStrategy(
13             provider="openai/gpt-4o-mini", # ou "ollama/llama3.2"
14             api_token=os.getenv("OPENAI_API_KEY"),
15             schema=ProduitInfo.model_json_schema(),
16             instruction="Extrais les produits de cette page"
17         )
18
19     async with AsyncWebCrawler() as crawler:
20         result = await crawler.arun(
21             url="https://shop.exemple.com",
22             config=CrawlerRunConfig(extraction_strategy=
23                                     extraction_strategy)
24         )
25         print(result.extracted_content) # JSON structure
26
27     asyncio.run(extraction_llm())

```

5.7 FireCrawl : API Cloud pour l'IA

FireCrawl est une API développée par Mendable.ai (Y Combinator S22) qui convertit n'importe quel site web en données structurées prêtes pour les LLMs. C'est une solution cloud qui gère automatiquement les proxies, l'anti-bot et le JavaScript.



Figure 5.4: FireCrawl : API de données web pour l'IA

5.7.1 Caractéristiques

- **96% de couverture web** : Sites JS-heavy et protégés
- **Extraction IA native** : Schémas Pydantic ou prompts naturels
- **3 modes** : Scrape (1 page), Crawl (site entier), Map (URLs)
- **Actions navigateur** : Clics, formulaires, scrolls automatisés

5.7.2 Installation et Exemple

```
1 pip install firecrawl-py
```

```
1 from firecrawl import Firecrawl
2
```

```

3     app = Firecrawl(api_key="fc-VOTRE_CLE_API")
4
5     # Scrape simple
6     result = app.scrape_url(
7         url="https://exemple.com/article",
8         formats=["markdown", "html"]
9     )
10    print(result["markdown"])
11
12    # Extraction en langage naturel
13    result = app.scrape_url(
14        url="https://shop.exemple.com/produit",
15        formats=[{
16            "type": "json",
17            "prompt": "Extrais le nom, prix et avis du produit"
18        }]
19    )
20    print(result["json"]) # Structure automatique

```

5.8 Jina Reader : Conversion Web Instantanée

Jina Reader est l'outil le plus simple pour convertir une URL en texte LLM-ready. Il suffit de préfixer une URL avec `https://r.jina.ai/` — aucune inscription, aucune clé API pour l'usage basique.



Figure 5.5: Jina Reader : Le plus simple pour commencer

5.8.1 Utilisation Sans Code

```

1     # Convertir une page en Markdown (navigateur ou curl)
2     https://r.jina.ai/https://exemple.com/article
3
4     # Recherche web
5     https://s.jina.ai/?q=python+web+scraping

```

5.8.2 Utilisation avec Python

```

1     import requests
2
3     def jina_reader(url: str) -> str:
4         """Convertir une URL en Markdown avec Jina Reader"""
5         reader_url = f"https://r.jina.ai/{url}"
6         response = requests.get(reader_url)
7         return response.text
8
9     # Exemple - gratuit, sans API key!
10    contenu = jina_reader("https://exemple.com/article")
11    print(contenu) # Markdown propre

```

5.9 Spider.cloud : Le Plus Rapide

Spider.cloud est le crawler le plus rapide du marché, construit en Rust (projet `spider-rs`). Il peut crawler 20,000 pages en quelques secondes.



Figure 5.6: Spider.cloud : Performances extrêmes

5.9.1 Exemple d'Utilisation

```

1  import requests
2  import os
3
4  API_KEY = os.environ["SPIDER_API_KEY"]
5
6  def spider_crawl(url: str, limit: int = 10) -> list:
7      response = requests.post(
8          "https://api.spider.cloud/crawl",
9          headers={
10             "Authorization": f"Bearer {API_KEY}",
11             "Content-Type": "application/json"
12         },
13         json={
14             "url": url,
15             "limit": limit,
16             "return_format": "markdown"
17         }
18     )
19     return response.json()
20
21     # Crawler 50 pages en quelques secondes
22     pages = spider_crawl("https://docs.exemple.com", limit=50)
23     for page in pages:
24         print(f"URL: {page['url']}")

```

5.10 Comparaison des Outils IA de Scraping

Critère	Crawl4AI	FireCrawl	Jina	Spider
Type	Open Source	API Cloud	API	API
Prix	Gratuit	Freemium	Gratuit+	Pay-per-use
Vitesse	Rapide	Moyenne	Très rapide	Record
Extraction LLM	Oui	Oui	Non	Beta
Deep Crawl	Oui	Oui	Non	Oui
Sans API Key	Oui	Non	Oui (basique)	Non

5.10.1 Guide de Choix

- **Pipeline RAG personnel** → Crawl4AI (gratuit, Markdown)
- **Prototype rapide** → Jina Reader (zéro config)
- **Production entreprise** → FireCrawl (fiabilité, support)
- **Grande échelle** → Spider.cloud (vitesse, coût/page)
- **Extraction complexe** → ScrapeGraphAI (logique LLM)
- **Budget zéro** → Crawl4AI + Ollama (100% local)

Licence IRM

Chapter 6

Flux Temps Réel (WebSockets)



Figure 6.1: Technologies de communication temps réel

6.1 Introduction aux WebSockets

Les WebSockets permettent une communication bidirectionnelle persistante entre client et serveur. Contrairement à HTTP qui fonctionne en requête-réponse, les WebSockets maintiennent une connexion ouverte pour l'échange continu de données.

6.1.1 Cas d'Utilisation

- **Données financières** : Cours boursiers, crypto-monnaies
- **Réseaux sociaux** : Notifications, feeds en temps réel
- **IoT** : Données de capteurs
- **Jeux en ligne** : État du jeu, positions des joueurs
- **Chat et messagerie** : Messages instantanés

6.1.2 Protocole WebSocket

- Connexion initiale via HTTP (handshake)
- Upgrade vers le protocole WebSocket
- Communication full-duplex sur une seule connexion TCP
- URLs en `ws://` ou `wss://` (sécurisé)

6.2 Implémentation avec Python

6.2.1 Installation

```
1 pip install websockets
```

6.2.2 Client WebSocket Simple

```
1 import asyncio
2 import websockets
3 import json
4
5 async def ecouter_flux():
6     """
7     Ecoute un flux WebSocket et affiche les messages.
8     """
9     uri = "wss://stream.exemple.com/data"
10
11     async with websockets.connect(uri) as websocket:
12         print("Connecte au flux")
13
14         try:
15             async for message in websocket:
16                 data = json.loads(message)
17                 print(f"Recu: {data}")
18
19         except websockets.exceptions.ConnectionClosed:
20             print("Connexion fermée")
21
22     # Executer
23     asyncio.run(ecouter_flux())
24
```

6.2.3 Client avec Reconnexion Automatique

```
1 import asyncio
2 import websockets
3 import json
4 from typing import Callable, Optional
5 import logging
6
7 logging.basicConfig(level=logging.INFO)
8 logger = logging.getLogger(__name__)
9
10
11 class WebSocketClient:
12     """
13     Client WebSocket robuste avec reconnexion automatique.
14     """
15
16     def __init__(
17         self,
18         uri: str,
19         on_message: Callable,
20         on_connect: Optional[Callable] = None,
21         on_disconnect: Optional[Callable] = None,
22         reconnect_interval: float = 5.0,
23         max_reconnect_attempts: int = 10
24     ):
```

```
25     self.uri = uri
26     self.on_message = on_message
27     self.on_connect = on_connect
28     self.on_disconnect = on_disconnect
29     self.reconnect_interval = reconnect_interval
30     self.max_reconnect_attempts = max_reconnect_attempts
31
32     self.websocket = None
33     self.running = False
34     self.reconnect_count = 0
35
36     async def connect(self):
37         """Etablit la connexion WebSocket."""
38         try:
39             self.websocket = await websockets.connect(
40                 self.uri,
41                 ping_interval=30,
42                 ping_timeout=10
43             )
44             self.reconnect_count = 0
45             logger.info(f"Connecte a {self.uri}")
46
47             if self.on_connect:
48                 await self.on_connect(self.websocket)
49
50         except Exception as e:
51             logger.error(f"Erreur de connexion: {e}")
52             raise
53
54     async def disconnect(self):
55         """Ferme la connexion."""
56         self.running = False
57         if self.websocket:
58             await self.websocket.close()
59
60         if self.on_disconnect:
61             await self.on_disconnect()
62
63     async def send(self, data: dict):
64         """Envoie des donnees au serveur."""
65         if self.websocket:
66             await self.websocket.send(json.dumps(data))
67
68     async def run(self):
69         """Boucle principale avec reconnexion automatique."""
70         self.running = True
71
72         while self.running:
73             try:
74                 await self.connect()
75
76                 async for message in self.websocket:
77                     if not self.running:
78                         break
79
80                     try:
81                         data = json.loads(message)
82                         await self.on_message(data)
```

```

83     except json.JSONDecodeError:
84         logger.warning(f"Message non-JSON: {message}")
85
86     except websockets.exceptions.ConnectionClosed as e:
87         logger.warning(f"Connexion fermée: {e}")
88
89     except Exception as e:
90         logger.error(f"Erreur: {e}")
91
92     # Reconnexion
93     if self.running:
94         self.reconnect_count += 1
95
96     if self.reconnect_count > self.max_reconnect_attempts:
97         logger.error("Nombre max de reconnections atteint")
98         break
99
100    logger.info(f"Reconnexion dans {self.reconnect_interval}s "
101               f"(tentative {self.reconnect_count})")
102    await asyncio.sleep(self.reconnect_interval)
103
104    async def __aenter__(self):
105        return self
106
107    async def __aexit__(self, *args):
108        await self.disconnect()
109
110
111    # Exemple d'utilisation
112    async def main():
113        messages_recus = []
114
115        async def traiter_message(data):
116            messages_recus.append(data)
117            print(f"Message: {data}")
118
119        async def a_la_connexion(ws):
120            # S'abonner a un canal
121            await ws.send(json.dumps({
122                "action": "subscribe",
123                "channel": "trades"
124            }))
125
126        client = WebSocketClient(
127            uri="wss://stream.exemple.com/ws",
128            on_message=traiter_message,
129            on_connect=a_la_connexion
130        )
131
132        # Executer pendant 60 secondes
133        try:
134            task = asyncio.create_task(client.run())
135            await asyncio.sleep(60)
136        finally:
137            await client.disconnect()
138
139        print(f"Total messages: {len(messages_recus)}")
140

```

```

141
142     asyncio.run(main())

```

6.2.4 Exemple : Flux Binance (Crypto)

```

1     import asyncio
2     import websockets
3     import json
4     from datetime import datetime
5     from collections import deque
6
7
8     class BinanceStreamClient:
9         """
10        Client pour les flux temps reel Binance.
11        """
12
13        BASE_URL = "wss://stream.binance.com:9443/ws"
14
15        def __init__(self, symbols: list, buffer_size: int = 1000):
16            self.symbols = [s.lower() for s in symbols]
17            self.buffer = deque(maxlen=buffer_size)
18            self.stats = {s: {"count": 0, "last_price": 0} for s in self.symbols}
19
20        def _build_url(self) -> str:
21            """Construit l'URL du flux combine."""
22            streams = [f"{s}@trade" for s in self.symbols]
23            return f"{self.BASE_URL}/{ '/' .join(streams)}"
24
25        async def process_trade(self, data: dict):
26            """Traite un evenement trade."""
27            symbol = data["s"]
28            price = float(data["p"])
29            quantity = float(data["q"])
30            timestamp = datetime.fromtimestamp(data["T"] / 1000)
31
32            trade = {
33                "symbol": symbol,
34                "price": price,
35                "quantity": quantity,
36                "value": price * quantity,
37                "timestamp": timestamp,
38                "is_buyer_maker": data["m"]
39            }
40
41            self.buffer.append(trade)
42            self.stats[symbol.lower()]["count"] += 1
43            self.stats[symbol.lower()]["last_price"] = price
44
45            # Afficher
46            direction = "SELL" if data["m"] else "BUY"
47            print(f"{timestamp.strftime('%H:%M:%S')} | {symbol} | "
48                  f"{direction} | {price:.2f} | {quantity:.4f}")
49
50        async def run(self, duration: int = None):
51            """

```

```

52     Execute le client.
53
54     Args:
55     duration: Duree en secondes (None = infini)
56     """
57     url = self._build_url()
58     print(f"Connexion a {url}")
59
60     start_time = asyncio.get_event_loop().time()
61
62     async with websockets.connect(url) as ws:
63         print("Connecte! En attente des trades...")
64
65         async for message in ws:
66             data = json.loads(message)
67
68             # Les flux combines ont une structure differente
69             if "stream" in data:
70                 await self.process_trade(data["data"])
71             else:
72                 await self.process_trade(data)
73
74             # Verifier la duree
75             if duration:
76                 elapsed = asyncio.get_event_loop().time() - start_time
77                 if elapsed >= duration:
78                     break
79
80             # Afficher les stats
81             print("\n--- Statistiques ---")
82             for symbol, stats in self.stats.items():
83                 print(f"{symbol.upper()}: {stats['count']} trades, "
84                       f"dernier prix: {stats['last_price']:.2f}")
85
86             def get_recent_trades(self, n: int = 10) -> list:
87                 """Retourne les n derniers trades."""
88                 return list(self.buffer)[-n:]
89
90
91             # Utilisation
92             if __name__ == "__main__":
93                 client = BinanceStreamClient(
94                     symbols=["BTCUSDT", "ETHUSDT"],
95                     buffer_size=10000
96                 )
97
98             # Collecter pendant 30 secondes
99             asyncio.run(client.run(duration=30))

```

6.2.5 Sauvegarde des Données en Temps Réel

```

1     import asyncio
2     import websockets
3     import json
4     import aiofiles
5     from datetime import datetime
6     from pathlib import Path

```

```

class StreamToFile:
    """
    Sauvegarde un flux WebSocket dans des fichiers.
    """

    def __init__(
        self,
        uri: str,
        output_dir: str,
        rotate_interval: int = 3600  # Rotation toutes les heures
    ):
        self.uri = uri
        self.output_dir = Path(output_dir)
        self.output_dir.mkdir(parents=True, exist_ok=True)
        self.rotate_interval = rotate_interval

        self.current_file = None
        self.file_handle = None
        self.message_count = 0
        self.last_rotation = None

    async def _get_file(self) -> aiofiles.threadpool.text.AsyncTextIOWrapper:
        """Obtient le fichier actuel, avec rotation si nécessaire."""
        now = datetime.now()

        # Verifier si rotation necessaire
        if self.last_rotation is None or \
            (now - self.last_rotation).seconds >= self.rotate_interval:

            # Fermer l'ancien fichier
            if self.file_handle:
                await self.file_handle.close()

            # Nouveau fichier
            filename = f"stream_{now.strftime('%Y%m%d_%H%M%S')}.jsonl"
            self.current_file = self.output_dir / filename
            self.file_handle = await aiofiles.open(self.current_file, "w")
            self.last_rotation = now

        print(f"Nouveau fichier: {self.current_file}")

        return self.file_handle

    async def run(self):
        """Execute la collecte."""
        async with websockets.connect(self.uri) as ws:
            print(f"Connecte, sauvegarde dans {self.output_dir}")

            async for message in ws:
                try:
                    data = json.loads(message)

                    # Ajouter timestamp de reception
                    data["_received_at"] = datetime.now().isoformat()

```

```
64     # Ecrire dans le fichier
65     file = await self._get_file()
66     await file.write(json.dumps(data) + "\n")
67
68     self.message_count += 1
69
70     if self.message_count % 1000 == 0:
71         print(f"{self.message_count} messages sauvegardés")
72         await file.flush()
73
74     except Exception as e:
75         print(f"Erreur: {e}")
76
77     async def close(self):
78         """Ferme les ressources."""
79         if self.file_handle:
80             await self.file_handle.close()
81
82
83     # Utilisation
84     async def main():
85         collector = StreamToFile(
86             uri="wss://stream.binance.com:9443/ws/btcusdt@trade",
87             output_dir="./data/trades",
88             rotate_interval=3600 # Nouveau fichier chaque heure
89         )
90
91         try:
92             await collector.run()
93         except KeyboardInterrupt:
94             print("\nArrêt...")
95         finally:
96             await collector.close()
97         print(f"Total: {collector.message_count} messages")
98
99
100     asyncio.run(main())
```

Chapter 7

Conclusion et Bonnes Pratiques

7.1 Récapitulatif des Mécanismes

Mécanisme	Cas d'usage	Outils Python
HTTP/HTTPS	Fichiers, datasets	requests, aiohttp
API REST	Services structurés	requests + auth
API GraphQL	Données précises	requests + query
Scraping statique	HTML sans JS	BeautifulSoup
Scraping dynamique	SPA, JavaScript	Selenium
Scraping large-scale	Volume important	Scrapy
WebSocket	Temps réel	websockets

7.2 Bonnes Pratiques Générales

1. **Respecter les sources** : robots.txt, rate limits, conditions d'utilisation
2. **Identifier votre bot** : User-Agent transparent avec contact
3. **Gérer les erreurs** : Retry avec backoff exponentiel
4. **Utiliser des timeouts** : Éviter les blocages
5. **Persister les données** : Sauvegardes régulières
6. **Logger les actions** : Traçabilité et débogage
7. **Tester avant production** : Valider sur échantillon
8. **Surveiller les changements** : Les sites évoluent

7.3 Arbre de Décision

Pour choisir le bon mécanisme :

1. **API disponible ?**
 - Oui → REST ou GraphQL
 - Non → Scraping
2. **Données temps réel ?**

- Oui → WebSocket
- Non → HTTP/API

3. **JavaScript requis ?**

- Oui → Selenium
- Non → BeautifulSoup

4. **Volume important ?**

- Oui → Scrapy ou async
- Non → requests simple

Licence IIRN

Chapter 8

Bibliographie et Ressources

8.1 Ouvrages de Référence

Pour approfondir vos connaissances, voici les ouvrages recommandés :

- **Mitchell, R.** (2018). *Web Scraping with Python: Collecting More Data from the Modern Web*, 2nd ed. O'Reilly Media. — La référence incontournable pour le scraping avec Python.
- **Sweigart, A.** (2019). *Automate the Boring Stuff with Python*, 2nd ed. No Starch Press. — Excellent pour débiter avec Python et l'automatisation.
- **Masse, M.** (2011). *REST API Design Rulebook*. O'Reilly Media. — Principes fondamentaux de conception d'APIs REST.
- **Banks, A. & Porcello, E.** (2018). *Learning GraphQL: Declarative Data Fetching for Modern Web Apps*. O'Reilly Media. — Introduction complète à GraphQL.
- **Richardson, L., Amundsen, M. & Ruby, S.** (2013). *RESTful Web APIs*. O'Reilly Media. — Conception et consommation d'APIs RESTful.
- **Perini, M., Padoan, L. & Vinciguerra, M.** (2024). *ScrapeGraphAI*. GitHub Open Source. — Bibliothèque Python de scraping utilisant les LLMs (GPT, Llama, etc.) pour une extraction intelligente et adaptative.
- **UncleCode** (2024). *Crawl4AI: Open-source LLM Friendly Web Crawler & Scraper*. GitHub (50k+ stars). — Crawler asynchrone optimisé pour RAG et pipelines IA.
- **Mendable.ai** (2024). *FireCrawl: The Web Data API for AI*. Y Combinator S22. — API cloud pour convertir sites web en données LLM-ready.
- **Jina AI** (2024). *Reader API*. Berlin. — Service de conversion URL vers Markdown, 100B+ tokens/jour.
- **A11yWatch** (2024). *Spider: The Fastest Web Crawling Service*. spider.cloud. — Crawler Rust haute performance (20,000+ pages/sec).

8.2 Documentation Officielle

Outil	URL	Description
Requests	requests.readthedocs.io	Client HTTP simple
aiohttp	docs.aiohttp.org	HTTP asynchrone
BeautifulSoup	crummy.com/software/BeautifulSoup	Parsing HTML
Selenium	selenium-python.readthedocs.io	Automatisation navigateur
Scrapy	docs.scrapy.org	Framework de scraping
ScrapeGraphAI	scrapegraph-doc.onrender.com	Scraping avec IA/LLMs
Crawl4AI	docs.crawl4ai.com	Crawler open source pour LLMs
FireCrawl	docs.firecrawl.dev	API cloud de scraping IA
Jina Reader	jina.ai/reader	Conversion URL → Markdown
Spider.cloud	spider.cloud/guides	Crawler rapide (Rust)
websockets	websockets.readthedocs.io	Client WebSocket
GraphQL	graphql.org/learn	Langage de requête
MDN HTTP	developer.mozilla.org/docs/Web/HTTP	Référence HTTP

8.3 Tutoriels Recommandés

- **Real Python** (realpython.com) — Tutoriels de qualité professionnelle :
 - Python’s Requests Library (Guide)
 - Beautiful Soup: Build a Web Scraper With Python
 - Python and REST APIs: Interacting With Web Services
 - Async IO in Python: A Complete Walkthrough
- **Mozilla Developer Network** (developer.mozilla.org) — Référence web complète
- **Google Developers** — Documentation sur robots.txt et bonnes pratiques

8.4 Sites d’Entraînement (Légaux)

Pour pratiquer le scraping de manière éthique :

- **httpbin.org** — Service pour tester les requêtes HTTP
- **jsonplaceholder.typicode.com** — API REST gratuite pour tests
- **scrapethissite.com** — Site conçu pour l’apprentissage du scraping
- **quotes.toscrape.com** — Site d’entraînement par les créateurs de Scrapy
- **books.toscrape.com** — Catalogue fictif pour pratiquer

8.5 APIs Publiques pour Pratiquer

- **GitHub API** (docs.github.com/en/rest) — REST et GraphQL
- **OpenWeatherMap** (openweathermap.org/api) — Données météo

- **JSONPlaceholder** (jsonplaceholder.typicode.com) — Données fictives
- **PokeAPI** (pokeapi.co) — Données Pokémon (ludique)
- **REST Countries** (restcountries.com) — Informations sur les pays
- **Binance API** (binance-docs.github.io/apidocs) — Données crypto temps réel

8.6 Aspects Légaux et Éthiques

- **CNIL** — La réutilisation des données publiquement accessibles en ligne (cnil.fr)
- **robots.txt** — Respecter les directives des sites (robotstxt.org)
- **RGPD** — Règlement européen sur la protection des données
- **CGU** — Toujours lire les Conditions Générales d'Utilisation des sites

Conseil pour les débutants : Commencez par les tutoriels Real Python, puis pratiquez sur les sites d'entraînement avant de vous attaquer à des projets réels. Respectez toujours les règles éthiques et légales du scraping.